

Note:

Machine learning libraries (PyTorch, TensorFlow, scikit-learn) usually expect inputs in the form: (number_of_samples, number_of_features)

Training Phase

1. Datasets (raw)

- PubChem: SMILES strings — variable-length text, e.g.
`"CC(=O)Oc1ccccc1C(=O)O"`
- CCLE/GDSC: omics profiles — typically a matrix of (n_cell_lines × n_genes), e.g. (1000, ~20000) for raw transcriptomics; IC50 values — scalar per (drug, cell line) pair
- STRING: protein interactions — edge list (n_edges, 2) of protein ID pairs, optionally with confidence scores (n_edges,)

2. Drug Encoding (Morgan Fingerprint)

- Input: SMILES string (1 per drug)
- Output: binary fingerprint vector, typically (2048,) or (1024,) per drug (bit vector)

Note: A binary fingerprint vector is a way of turning a chemical molecule into a fixed-length sequence of 0s and 1s that a machine learning model can understand.

[0, 1, 0, 0, 1, 1, 0, ..., 1]

If it's a 2048-bit fingerprint, there are 2048 positions.

Each position (bit) represents the presence or absence of a particular molecular substructure or pattern.

3. Omics Preprocessing (Normalisation → Imputation → PCA)

- Input: raw omics matrix (n_cell_lines, ~20000) (genes/features)
- After PCA compression, output: (n_cell_lines, k) where k is often 100–500 depending on variance retained

Note: Raw omics input (e.g., transcriptomics or genomics) starts with an enormous number of features—typically **~20,000 genes per cell line**. Passing 20,000 features directly into a Graph Neural Network (GNN) leads to two main issues:

1. **Curse of Dimensionality:** Extreme overfitting due to high noise and redundant gene interactions.
2. **Computational Strain:** High GPU memory consumption and slow training times.

PCA compresses those 20,000 continuous gene expression features down to a dense vector of size **k**

4. PPI Graph Loading

- Input: protein interaction edge list (**n_edges**, 2)
- Output: PPI node features — could be identity/one-hot (**n_proteins**, **n_proteins**) or learned embeddings (**n_proteins**, **d**), e.g. (**n_proteins**, 128)

Note: One-hot and learned embeddings are two ways of representing proteins here.

Suppose there are four proteins: TP53 EGFR AKT1 BRCA1

If we used a one-hot representation, it would look like

TP53 = [1 0 0 0], EGFR = [0 1 0 0], AKT1 = [0 0 1 0], BRCA1 = [0 0 0 1]

The problem with this representation is that it's huge and doesn't capture any biological similarity between proteins.

Learned embeddings is different:

Instead of a giant one-hot vector, every protein starts with a compact embedding.

Think of it like describing every protein using 128 hidden characteristics.

Not things humans explicitly define like:

- "tumour suppressor"
- "kinase"
- "DNA repair"

Instead, the model discovers useful numerical features during training.

TP53 =>

[0.82

-0.15

0.41

...]

might encode properties that help predict drug response, even though we can't directly interpret each number.

5. Fusion (Cross-Attention)

- Input: drug embedding (2048,) or projected to (d,), e.g. (128,); omics embedding (k,) projected to (d,), e.g. (128,)
- Output: fused vector, typically same dimension as the attention hidden size, e.g. (128,) or (256,) per drug-cell line pair

Note: Before fusion, your representations exist in completely different formats:

- **The Drug Embedding:** Encodes chemical structure (atoms, functional groups, rings).
- **The Omics Embedding (\$k\$):** Encodes biological state (gene expression levels, mutations).

You can't directly compare "chemical ring structures" to "gene expression levels"—it's like comparing apples to oranges.

Therefore, both the drug and omics embeddings are projected into a common 128-dimensional embedding space before fusion.

6. Training Graph (G = (V, E))

- Drug Nodes: (n_drugs, d) e.g. (n_drugs, 128)
- Cell Line Nodes: (n_cell_lines, d) e.g. (n_cell_lines, 128)
- Protein Nodes: (n_proteins, d) e.g. (n_proteins, 128)
- Edge index: (2, n_edges) (standard PyTorch Geometric format)

Note: Why does Edge index have the shape (2, n_edges)?

Suppose you have 4 nodes:

0 = Drug A

1 = Protein TP53

2 = Protein EGFR

3 = Cell Line X

Assume these connections exist:

Drug A → TP53

TP53 → EGFR

EGFR → Cell Line X

The edge index is

edge_index = [[0, 1, 2], [1, 2, 3]]

Which means:

Each **column** represents **one edge**.

	Edge 1	Edge 2	Edge 3
Source	0	1	2
Target	1	2	3

We could have done the other way around but PyTorch Geometric expects the transpose.

This layout is more efficient for GPU operations because PyG frequently needs to access **all source nodes** or **all target nodes** at once during message passing.

7. Model Training

- GAT Layers input: node features (n_nodes, d_in) + edge index $(2, n_edges)$
- GAT Layers output: updated node embeddings (n_nodes, d_hidden) , e.g. $(n_nodes, 256)$ (or $d_hidden \times n_heads$ if multi-head attention is concatenated)
- Pair embeddings (drug-cell line pair, concatenated or combined): $(batch_size, 2 \times d_hidden)$ or $(batch_size, d_hidden)$
- MLP Head output: $(batch_size, 1)$ — a single scalar IC50/response prediction per pair
- Trained Model Weights: saved as a state dict — not a "shape" per se, but a collection of weight tensors (e.g. GAT layer weights (d_in, d_out) per layer, MLP weights $(d_hidden, 1)$, etc.)

Inference Phase

8. User Input (Cell Line CSV)

- Proteomics: $(1, n_proteins)$ for a single new cell line
- Genomics: $(1, n_genes)$
- Transcriptomics: $(1, n_transcripts)$

The **1** simply means

"I'm predicting for one new sample."

9. Inference Preprocessing

- Input: raw single-sample vectors as above
- Output: normalised + PCA-compressed vector $(1, k)$, matching training's k (e.g. $(1, 100)$)

Your model was **not trained on raw data**.

During training you performed preprocessing:

Therefore, if you skip preprocessing during inference, the model receives data in a completely different format than it learned from.

Why normalisation?

Imagine two genes.

Gene A 0–10

Gene B 0–100,000

If you feed them directly into a neural network,

Gene B dominates simply because its numbers are much larger.

Normalisation rescales everything to a comparable range.

For example,

Before

Gene A = 5

Gene B = 42000

After

Gene A = 0.52

Gene B = 0.61

Now neither feature overwhelms the other.

10. Inference Fusion

- Input:
- omics vector $(1, k)$ projected to $(1, d)$;
- drug embedding $(1, d)$ for the query drug
- Output: fused vector $(1, d)$, e.g. $(1, 128)$

Note: Projection means transforming

Why make them both $(1, 128)$?

Imagine trying to compare:

Drug: 2048 numbers

Cell line: 100 numbers

If they were the same size, i.e. after projection they both turn into 128, they can be compared and combined.

Note: Step basically does “Take the information about the cell line and the information about the drug, and combine them into one representation that the model can use for prediction.”

11. Inference Graph (Build User Graph)

- Output: small graph — 1 new cell line node (1, d) connected to existing drug/protein nodes, edge index (2, n_new_edges)

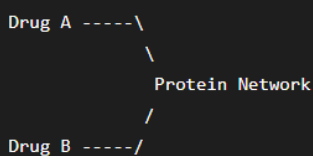
After preprocessing, you have a compressed embedding for the new cell line.

But the GAT doesn't operate on isolated vectors.

It operates on **graphs**.

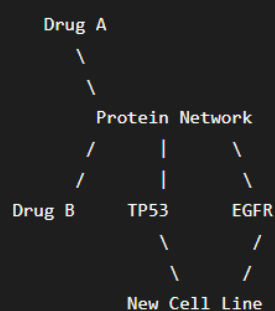
So you have to place this new cell line into the graph.

Imagine your training graph looked like this:



Now a new patient arrives.

You insert them into the graph.



The graph is now saying:

"Here is a new cell line. Here are the proteins and drugs it is related to.

12. Prediction

- GAT Forward Pass output: $(1, d_hidden)$ embedding for the new cell line/drug pair
- MLP Forward Pass output: $(1, 1)$ — single scalar prediction

- ❖ The **Graph Attention Network (GAT)** receives the new cell line node and its connected graph.
- ❖ The GAT **passes information between connected nodes**, allowing the cell line to learn from relevant proteins (and drugs, depending on the graph design).
- ❖ **Attention weights** determine which neighbouring nodes are most influential.
- ❖ The GAT outputs an **updated embedding** that captures both the cell line's own omics information and its biological context.
- ❖ The updated embedding is fed into a **Multi-Layer Perceptron (MLP)**.
- ❖ The MLP transforms the embedding into a **single scalar prediction**.
- ❖ The output is the **predicted drug response (IC50)** for the queried drug-cell line pair.

An intuitive analogy is joining a new person to a social network:

- Before Step 11, the new person exists but **knows nobody**.
- Step 11 introduces them into the network by connecting them to other people.
- Step 12 is when conversations happen, allowing the new person to learn from their connections before the final decision is made.

13. Output

- Predicted drug response / IC50: a single float value, e.g. $IC50 = 4.32$ (μM or log-scale depending on your target transform)